

Rapport Global — Itération 2

Projet Long IDE7 · Mini IDE Java avec analyse UML, génération de code et autocomplétion

Équipe : Mouad Mardy, ESMAHI Salah eddine
Ziani Mohamed Amine, Jaouad Ait Nasser
Yahya El Morhder, Yasser Errerhaoui
Ahmed El Fanaoui, Saad Lefrais

Encadrant : Guillaume Dupont

Filière : SN — ENSEEIHT

Date : 9 mai 2026

1. Introduction générale

L'itération 1 avait permis de poser les fondations du projet : modèle UML interne (`JavaClass`, `JavaAttribut`, `JavaMethode`, `JavaRelation`), parseur Java basé sur des expressions régulières, première version de l'interface JavaFX, moteur de compilation et d'exécution via `ProcessBuilder`, et structure `Trie` pour l'autocomplétion.

L'itération 2 avait un objectif clair : transformer ces bases en modules réellement utiles, corriger les limites identifiées et amorcer les premières intégrations entre composants. Chacun des quatre sous-groupes a travaillé de façon autonome sur son périmètre tout en exposant des interfaces exploitables par les autres groupes.

Ce rapport synthétise les contributions de l'ensemble de l'équipe.

2. Module UML — Analyse et génération de code Java

Auteurs : Mouad Mardy · ESMAHI Salah eddine

2.1. Contexte de l'itération 2

En itération 1, le binôme avait mis en place le modèle interne et les premières fonctionnalités : parseur Java textuel, affichage UML en console, et générateur de code Java produisant attributs, constructeurs, getters et setters. La classe `JavaRelation` existait mais n'était pas encore exploitée. L'itération 2 visait à combler cela : gérer les relations entre classes, exporter le code généré vers un fichier, et connecter le générateur à l'interface JavaFX.

2.2. Mouad Mardy — Analyse Java et affichage UML

Gestion des relations dans le parseur

Le parseur `JavaParser` a été enrichi pour détecter deux types de relations directement dans le code source :

- **Héritage** via le mot-clé `extends` : une relation de type `INHERITANCE` est créée entre la classe analysée et sa classe parente.
- **Implémentation** via le mot-clé `implements` : une ou plusieurs relations de type `REALIZATION` sont créées. Le cas de l'implémentation multiple est géré (liste de cibles).

Par exemple, pour le code suivant :

```
public class Etudiant extends Personne implements Comparable {}
```

le parseur produit deux relations : `Etudiant` → `Personne` (`INHERITANCE`) et `Etudiant` → `Comparable` (`REALIZATION`).

Amélioration de l'affichage UML textuel

La classe `UMLPrinter` affiche désormais les relations détectées en complément des attributs et méthodes, avec les symboles UML standards (`-|>` pour l'héritage, `..>` pour l'implémentation). Cela permet de visualiser la structure complète d'un programme directement dans le terminal.

Des tests ont été réalisés sur des classes Java simples pour valider l'extraction et l'affichage des relations.

● **Avancement : 100%**

2.3. ESMAHI Salah eddine — Génération de code Java

Gestion des relations dans le générateur

`JavaCodeGenerator` a été étendu pour prendre en compte les relations présentes dans le modèle :

- La méthode `appendDeclaration` génère maintenant `extends NomClasse` si une relation `INHERITANCE` est présente.
- Elle accumule les cibles `REALIZATION` dans une liste pour générer `implements Interface1, Interface2, ...`, car `extends` est unique mais `implements` peut être multiple.
- Les relations de composition et d'agrégation sont reflétées comme attributs dans le corps de la classe.

Export vers fichier .java

La classe `JavaFileExporter` a été implémentée : elle prend la chaîne produite par `JavaCodeGenerator` et l'écrit dans un fichier `.java` sur le disque via `java.nio.file.Files`. Le nom du fichier est automatiquement déduit du nom de la classe.

Résolution du conflit Git

Un conflit sur `JavaClass.java` est survenu lors du merge : la branche de Mouad avait introduit `addRelation()` sans synchronisation préalable. Le conflit a été résolu manuellement en conservant la version enrichie, puis commité et poussé :

```
git add JavaClass.java
git commit -m "fix: resolution conflit merge - ajout addRelation()"
git push origin ma-branche
```

Intégration avec l'interface JavaFX

Le bouton « Générer le code » de l'interface transmet désormais la classe sélectionnée dans le diagramme à `JavaCodeGenerator`. Le résultat s'affiche dans une `TextArea` et est exporté sur le disque via `JavaFileExporter`. La synchronisation en temps réel entre le diagramme et le code n'est pas encore implémentée et est reportée à l'itération 3.

• **Avancement : 100% sur les objectifs de l'itération 2. Synchronisation temps réel reportée.**

2.4. Limites actuelles du module

- L'analyse repose encore sur des expressions régulières : les structures Java complexes (lambdas, génériques imbriqués) ne sont pas couvertes.
- L'affichage UML reste textuel : la visualisation graphique est reportée à l'itération 3.
- La synchronisation temps réel diagramme ↔ code n'est pas encore implémentée.

3. Interface graphique JavaFX

Auteurs : Ziani Mohamed Amine · Jaouad Ait Nasser

Période couverte : 9 avril 2026 – 8 mai 2026

3.1. Contexte de l'itération 2

En itération 1, une base fonctionnelle existait : création et ouverture de projet, éditeur de texte simple. L'itération 2 visait à structurer l'interface selon un pattern MVC propre, à stabiliser la navigation entre écrans, et à enrichir significativement l'éditeur et la gestion des fichiers.

3.2. Architecture mise en place

L'interface repose sur :

- **Pattern MVC strict** : séparation claire entre `controllers`, `views` et `models`.
- **JavaFX 17** comme framework graphique.
- **Ikonli** (Ant Design Icons) pour les icônes.
- **Persistance locale** via `~/ide7/projects.json`.

Les classes principales sont : `AppController` (point d'entrée, gestion des écrans), `MainController` (éditeur principal), `MainView`, `WelcomeController`, `NewProjectController`, `OpenProjectController`, `ProjectStore`, `ProjectEntry`.

3.3. Fonctionnalités réalisées

Navigation entre écrans

- Écran d'accueil avec liste des projets récents.
- Écran de création de projet avec validation.
- Écran d'ouverture de projet avec bouton « Nouveau projet » intégré.
- Retour vers l'accueil depuis l'éditeur.

Éditeur

- Explorateur de fichiers arborescent avec icônes distinctes pour dossiers et fichiers.
- Édition de fichiers `.java` et `.txt`.
- Actions d'édition complètes : Undo, Redo, Cut, Copy, Paste, Select All.
- Recherche simple, remplacement simple et remplacement global.
- Sauvegarde manuelle et auto-save.
- Barre de menus et barre d'outils avec icônes, uniformisées en français.

Gestion des fichiers

- Création de fichiers Java selon leur type : classe, classe abstraite, interface, enum, annotation.
- Création de packages avec validation du nom.
- Suppression de fichiers et dossiers.
- Import par glisser-déposer.
- Sécurisation des opérations pour rester dans le répertoire du projet.

3.4. Difficultés et solutions

- **Bug de sélection de texte** : corrigé pendant l'itération.
- **Validation des noms Java** : contrôles renforcés pour les noms de classes et de packages.

- **Sécurisation des opérations fichiers** : ajout de vérifications pour empêcher les opérations hors du répertoire projet.

- **Avancement : 100% sur tous les objectifs de l'itération 2.**

3.5. Limites actuelles

- Coloration syntaxique encore basique.
- Absence de raccourcis clavier avancés.
- Préférences utilisateur non persistantes.
- Pas encore de gestion multi-onglets.

4. Module Debug & Exécution

Auteurs : Ahmed El Fanaoui · Saad Lefrais

4.1. Contexte de l'itération 2

En itération 1, le module couvrait : `CompilerService` (compilation via `javac`), `ExecutionService` (exécution via `java`), `ExecutionResult` (encapsulation des résultats), `CompilationErrorParser` (parsing des erreurs de compilation) et une première version de `MemorySnapshot`. La principale limite identifiée était la fusion des flux `stdout` et `stderr` via `redirectErrorStream(true)`, qui empêchait tout parsing fiable des exceptions runtime.

4.2. Architecture après itération 2

Cinq responsabilités dans le paquet `fr.inptoulouse.sn.ide7.debug` :

- **Compilation** : `CompilerService`, `CompilationError`, `CompilationErrorParser`.
- **Exécution (modifié)** : `ExecutionService`, `ExecutionResult`, `enum TerminationReason` (`NORMAL`, `KILLED_BY_USER`, `TIMEOUT`).
- **Analyse runtime (nouveau)** : `RuntimeExceptionInfo`, `RuntimeExceptionParser`.
- **Suivi mémoire** : `MemorySnapshot`, `MemorySnapshotManager`.
- **Façade (modifié)** : `DebugController`.

4.3. Ticket SCRUM-33 — Capture des exceptions runtime

Réalisé par Saad Lefrais.

La séparation `stdout/stderr` est la principale amélioration technique de cette itération. Deux threads dédiés lisent désormais les flux en parallèle, ce qui supprime le risque de blocage de buffer signalé en itération 1 et permet un parsing fiable de la sortie d'erreur.

`RuntimeExceptionParser` reconnaît les en-têtes `Exception in thread "..."` et `Caused by:`, collecte les lignes `at ...` qui suivent, et construit des objets `RuntimeExceptionInfo`

contenant le type de l'exception, le message et la stack trace. Testé sur `NullPointerException`, `ArrayIndexOutOfBoundsException`, et sur des entrées vides ou nulles.

- **Avancement : 100%**

4.4. Ticket SCRUM-34 — Arrêt d'un programme bloqué

Réalisé par Saad Lefrais.

Un timeout configurable et une méthode `stopExecution()` thread-safe ont été implémentés. Le bouton « Arrêter » est cliqué sur le thread Swing ; `execute()` tourne sur un autre thread. La référence au `Process` courant est stockée dans un `AtomicReference<Process>`, ce qui assure la cohérence sans verrou explicite.

- **Avancement : 100%**

4.5. Ticket SCRUM-26 — Intégration dans l'IHM

Réalisé par Ahmed El Fanaoui.

La triade Compiler / Exécuter / Arrêter est connectée à l'interface graphique avec gestion d'état : les boutons sont grisés selon que le programme tourne ou non, ce qui évite les appels redondants.

- **Avancement : 100%**

4.6. Choix de conception notables

- **Rétrocompatibilité** : l'ancien constructeur `ExecutionResult(success, exitCode, output)` et la signature `DebugController.runFile()` sont conservés. `CompilerService` n'a pas été modifié.
- **Thread-safety** : usage de `AtomicReference<Process>` plutôt qu'un verrou explicite, pattern suggéré par un assistant IA et validé par l'équipe.
- **Robustesse du parser** : une expression régulière buggée produite par l'IA n'a été détectée qu'à l'exécution des tests, ce qui a conduit l'équipe à systématiser la validation avant intégration.

4.7. Méthode agile

Backlog priorisé sur Jira (colonnes Idée / À faire / En cours / En revue / Terminé). Branches Git dédiées au sous-groupe (`debugg/debug`), commits atomiques préfixés `debug: SCRUM-XX`. Revue de code croisée avant chaque fusion.

- **Avancement global du module : 100% sur les trois tickets retenus.**

4.8. Limites actuelles

- Pas encore de console interactive avec entrée standard (`stdin`).
- Compilation limitée à un fichier unique.

- Mode pas-à-pas et breakpoints non implémentés.
- Visualisation graphique des variables non implémentée.

5. Module d'autocomplétion

Auteurs : Yahya El Morhder · Yasser Errerhaoui

5.1. Contexte de l'itération 2

En itération 1, le module extrayait les mots-clés d'un fichier Java, les indexait dans un `Trie`, et proposait des suggestions classées par fréquence. L'itération 2 visait à rendre ce moteur réellement utile : indexation récursive, scoring plus pertinent, interface interactive enrichie, et architecture prête pour l'intégration JavaFX.

5.2. Architecture du module

- `Completer` : point d'entrée interactif.
- `AutocompleteEngine` : moteur de suggestion (nouveau).
- `KeywordExtractor` : extraction et indexation des identifiants.
- `Trie` / `TrieNode` : structure de recherche par préfixe.
- `Suggestion` : représentation d'une proposition avec valeur, fréquence, score et origine (nouveau).
- `IndexStats` : statistiques d'indexation (nouveau).

5.3. Améliorations apportées

Indexation

- Indexation récursive d'un dossier complet (fichiers `.java`, `.txt`, `.md`).
- Extraction plus robuste : découpage des identifiants composés (`buildTrie` → `build`, `Trie`).
- Ajout de mots-clés Java utiles même s'ils n'apparaissent pas dans la source analysée.

Classement des suggestions

Le score combinant fréquence et proximité au préfixe remplace le simple tri par fréquence de l'itération 1. Cela améliore la pertinence des propositions pour les identifiants rares mais récents.

Interface interactive

De nouvelles commandes ont été ajoutées au mode interactif : `:help`, `:stats`, `:files`, `:reindex`, `:limit`, `:source`. Ces commandes permettent de consulter les statistiques d'indexation, de changer la source à analyser et de réindexer à la volée.

5.4. Validation

- Tests JUnit sur `Trie` (`TrieTest`) et sur `KeywordExtractor` (`KeywordExtractorTest`).
- Tests sur `AutocompleteEngine` avec vérification des suggestions obtenues.
- Session interactive validée manuellement sur `Trie.java`.

• **Avancement : 100% sur les objectifs de l'itération 2.**

5.5. Limites actuelles

- Pas de prise en compte du contexte syntaxique précis de la ligne de code.
- Pas de persistance de l'index entre deux sessions.
- Heuristique de classement encore simple.
- Pas encore d'intégration directe dans l'interface JavaFX.

6. Utilisation de l'intelligence artificielle

Les quatre sous-groupes ont eu recours à des assistants IA (Claude, ChatGPT) pendant cette itération, principalement pour :

- **Sélection des tickets** : aide au classement par complexité et dépendances.
- **Génération de code boilerplate** : parsers, getters, modèles, patterns (ex. : `AtomicReference` pour le thread-safety dans le module `Debug`).
- **Correction d'erreurs** : syntaxe Java, organisation du modèle interne.
- **Documentation** : aide à la rédaction des rapports et de la documentation technique.

Dans tous les cas, l'usage a été encadré et critique. Le module `Debug` a notamment détecté une expression régulière buggée produite par l'IA (`S+?`), qui n'a été corrigée qu'après exécution des tests unitaires. L'IA a joué un rôle de « pair-programming augmenté » : elle accélère, mais ne dispense pas de relire, tester et exercer son propre jugement. Toutes les décisions de conception ont été prises par les équipes.

7. Bilan global de l'itération 2

7.1. Avancement par module

Module	Objectifs itération 2	Avancement
UML (Mouad)	Relations <code>extends</code> / <code>implements</code> dans parseur + affichage	100%
UML (Salah)	Relations dans générateur, export <code>.java</code> , intégration JavaFX	100%
UI/JavaFX	Navigation, éditeur complet, gestion fichiers, pattern MVC	100%
Debug (Ahmed)	Intégration IHM Compiler/Exécuter/Arrêter	100%
Debug (Saad)	Exceptions runtime, séparation flux, timeout + arrêt thread-safe	100%
Autocomplétion	Indexation récursive, scoring, interface interactive, nouvelles classes	100%

7.2. Ce qui est fonctionnel à l'issue de l'itération 2

- Analyse Java avec détection de `extends` et `implements`.
- Affichage UML textuel complet avec relations.
- Génération de code Java avec relations, export vers fichier `.java`.
- Interface JavaFX stable : navigation fluide entre écrans, éditeur complet, gestion de fichiers (création, suppression, import).
- Bouton « Générer le code » connecté au générateur UML.
- Compilation + exécution avec capture structurée des exceptions runtime.
- Arrêt forcé thread-safe des programmes bloqués.
- Triade Compiler/Exécuter/Arrêter dans l'IHM avec gestion d'état.
- Moteur d'autocomplétion avec indexation récursive, scoring et interface interactive enrichie.

7.3. Reporté à l'itération 3

- **UML** : visualisation graphique des diagrammes (JavaFX), synchronisation temps réel UML ↔ code Java, tests JUnit formels sur `JavaCodeGenerator`.
- **Debug** : console interactive avec `stdin` (SCRUM-32), compilation multi-fichiers (SCRUM-36), mode pas-à-pas (SCRUM-28), breakpoints (SCRUM-27), visualisation graphique des variables (SCRUM-29/SCRUM-35).
- **Autocomplétion** : intégration dans l'interface JavaFX, indexation incrémentale, suggestions provenant du JDK.
- **UI** : tests fonctionnels automatisés, finalisation des menus désactivés, préparation de la démonstration finale.

8. Conclusion

L'itération 2 a fait franchir un cap réel au projet IDE7. Chaque module est passé d'une base démonstrative à un composant fonctionnel : les relations UML sont détectées et générées, l'interface est stable et utilisable, le moteur de compilation gère les exceptions runtime et l'arrêt forcé, et l'autocomplétion propose des suggestions pertinentes sur un dossier entier.

Les premières intégrations entre modules ont été amorcées (bouton « Générer le code », triade Compiler/Exécuter/Arrêter dans l'IHM). L'itération 3 devra consolider ces connexions et développer les fonctionnalités avancées : visualisation graphique UML, console interactive, mode pas-à-pas, et intégration complète de l'autocomplétion.